

Algorithmization and Programming of Solutions

Part I

TOPIC #8

SUBPROGRAMS AND FUNCTIONS

RTU LECTURER OLGA YAKOVLEVA, MG.SC.ING

Table of contents

1. Subprograms and functions:

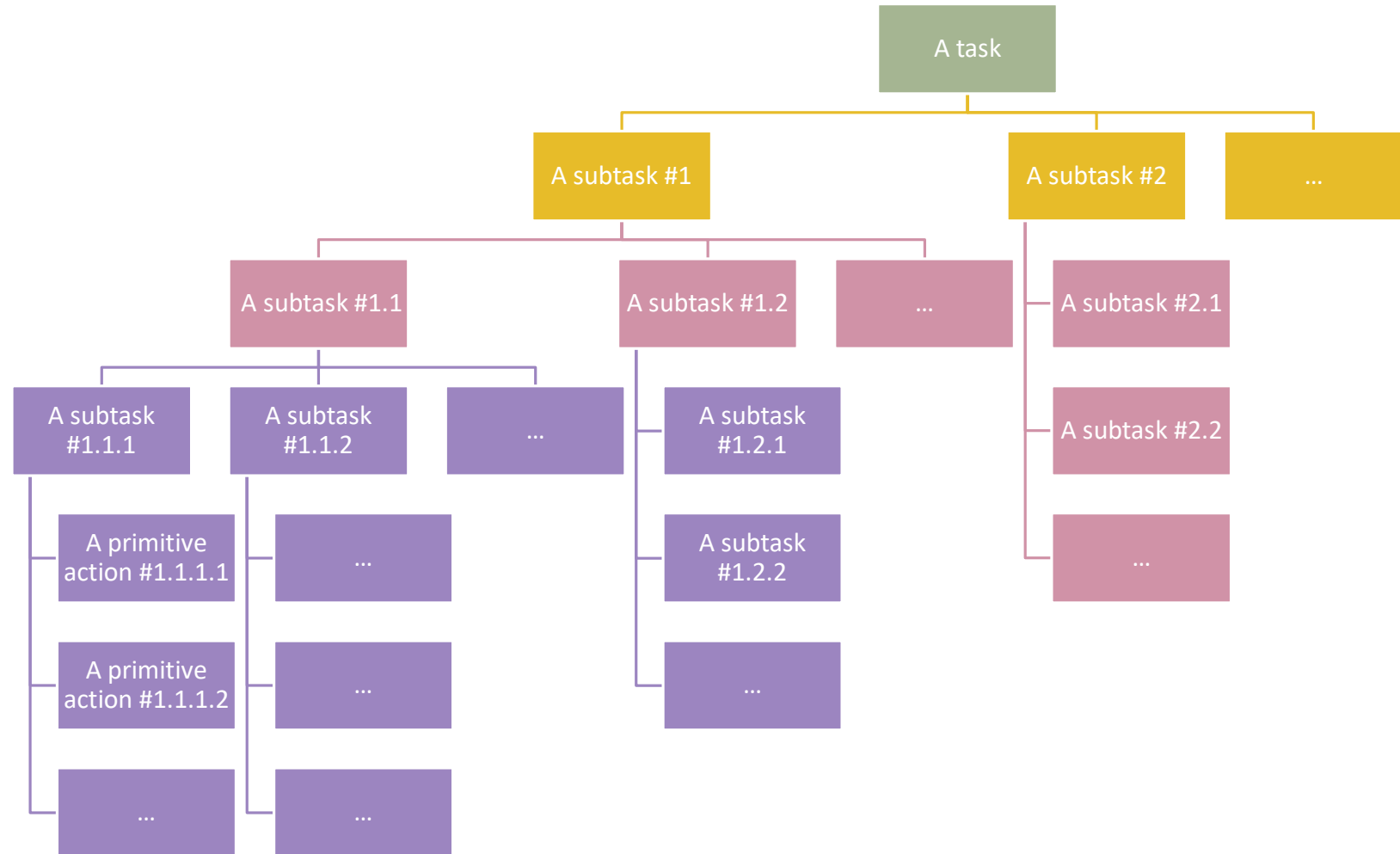
- a concept of a subprogram
- types of subprograms
- the advantages of using subprograms
- functions in programming language C
- the function main()
- global and local variables

1. Subprograms and functions

A CONCEPT OF A SUBPROGRAM, TYPES OF SUBPROGRAMS, THE ADVANTAGES OF USING SUBPROGRAMS, FUNCTIONS IN C, THE MAIN() FUNCTION, GLOBAL AND LOCAL VARIABLES

Subprograms (1/2)

A process of program writing usually involves decomposing of a task into subtasks:



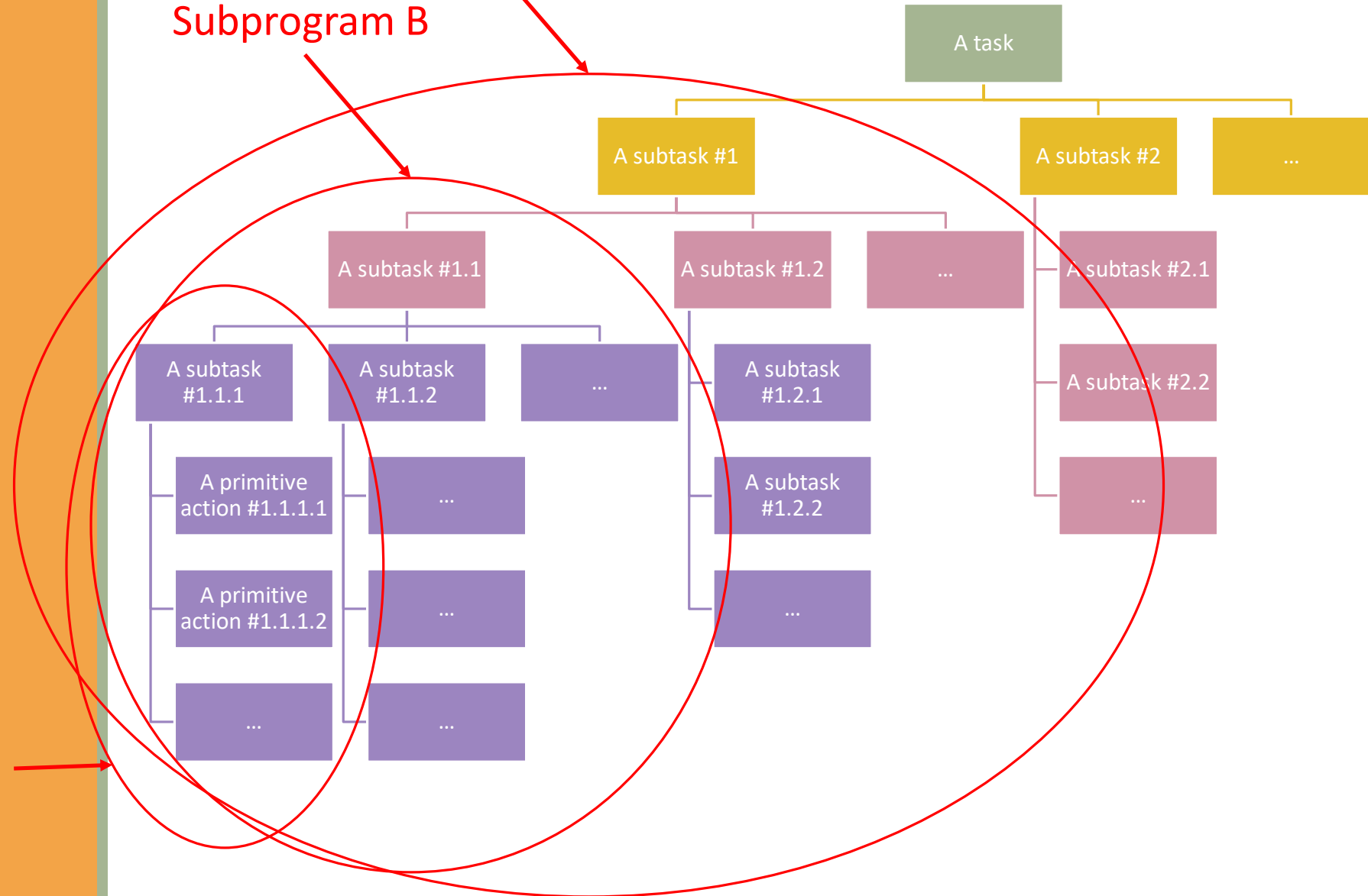
Subprograms (2/2)

When a task is decomposed, subtasks can be joined into subprograms

Subprogram A

Subprogram B

Subprogram C



Types of subprograms

Depending on a programming language types of subprograms can be the following:

- **a subroutine** = a subprogram
- **a procedure** – a subprogram that doesn't have a return value
- **a function** – a subprogram that has a return value
- **a method** = a procedure or a function

In programming language C all subprograms are called functions

Advantages of using subprograms

1. They help to keep a program simple, and, thus, more readable (*modularity*)
2. They allow a programmer to use the same code as many times as needed throughout a program (*modularity*)
3. They allow a programmer to create his/her own functions (*extensibility*)
4. They can be used in other programs (*reusability*)
5. They allow to change a program, by changing the body of a subprogram (*maintainability*)

Functions in C

C program always includes at least one function

If there is only one function, the name of it should be **main()**, because an execution of the program always starts with calling `main()` function

A function processes data that can be stored in the body of this function or can be passed to the function from the main program

If data are passes to the function, then it should have **arguments** that are listed in parentheses; if there are two or more arguments, they are separated by comma

The function arguments are identifiers with a specified data type

Even if there are no arguments in the function, parantheses remain

The function body and a return value

Actions that are performed by the function are listed in its **body**, i.e., in braces after the arguments

Usually the function performs some computations, that's why it has a so called **return value**, where the computation result is stored

The type of the return value is strictly defined before the name of function

```
type name (arg1, arg2, ... argN) {  
    action1;  
    action2;  
    ...  
    actionM;  
}
```

arguments

body

Examples of functions, I

```
double power(double val, unsigned pow) {  
    double ret_val = 1.0;  
    unsigned i;  
    for(i = 0; i < pow; i++)  
        ret_val *= val;  
    return(ret_val);  
}
```

Examples of functions, II

```
// no return value
```

```
void print_happy_birthday(int age) {  
    printf("Congratulations on your %dth  
        Birthday\n", age);  
  
    return;  
}
```

Examples of functions, III (1/2)

```
#include <stdio.h>
/* function declaration */
int max(int num1, int num2);
int main () {
    /* local variables*/
    int a=100; int b=200; int ret;
    /* calling the function */
    ret = max(a, b);
    printf("Max value is: %d\n", ret );
    return 0;
}
```

Examples of functions, III (2/2)

```
/* function description */  
int max(int num1, int num2) {  
    /* local variables */  
    int result;  
    if (num1 > num2) result = num1;  
    else result = num2;  
    return result;  
}
```

Passing arguments to the function (1/2)

In the declaration of the function only types and identifiers of arguments are mentioned; these arguments have no value and are called **formal**

When calling the function, constants, variables or even expressions are inserted in place of the arguments; now these arguments have values and are called **actual**

Before an execution of the function free memory is allocated for storing the values of actual parameters (*dynamic memory allocation*)

This memory is disposed only after the execution of the operator **return**

Passing arguments to the function (2/2)

BY VALUE (X AND Y VALUES DON'T CHANGE)

```
void swap(int a, int b) {  
    int t=a;  
    a=b;  
    b=t;  
}  
  
void main() {  
    int x=5, y=7;  
    printf("Before: x=%d, y=%d\n",x,y);  
    swap(x,y);  
    printf("After: x=%d, y=%d\n",x,y);  
}
```

BY ADDRESS (X AND Y VALUES CHANGE)

```
void swap(int *a, int *b) {  
    int t=*a;  
    *a=*b;  
    *b=t;  
}  
  
void main() {  
    int x=5, y=7;  
    printf("Before: x=%d, y=%d\n",x,y);  
    swap(&x,&y);  
    printf("After: x=%d, y=%d\n",x,y);  
}
```

Passing an one-dimensional array to the function (1/2)

```
#include <stdio.h>
float average(float a[]);

int main() {
    float avg,
    c[]={23.4, 55, 22.6, 3, 40.5, 18};
    avg=average(c);
    printf("Average age=%.2f", avg);
    return 0;
}
```


Passing an one-dimensional array to the function (2/2)

```
float average(float a[]) {  
    int i;  
    float avg, sum=0.0;  
    for (i=0; i<6; ++i) {  
        sum+=a[i];  
    }  
    avg = (sum/6);  
    return avg;  
}
```

Passing a two-dimensional array to the function (1/2)

```
#include <stdio.h>
void Function(int c[2][2]);
int main() {
    int c[2][2], i, j;
    printf("Enter 4 numbers:\n");
    for (i=0; i<2; ++i)
        for (j=0; j<2; ++j) {
            scanf("%d", &c[i][j]);
        }
    Function(c);
    return 0;
}
```

Passing a two-dimensional array to the function (2/2)

```
void Function(int c[2][2]) {  
    /* Instead of the above line,  
    void Function(int c[][2]){ is also valid */  
    int i,j;  
    printf("Displaying:\n");  
    for (i=0;i<2;++i)  
        for (j=0;j<2;++j)  
            printf("%d\n",c[i][j]);  
}
```

Passing an one-dimensional array to the function as a pointer (1/2)

```
#include <stdio.h>
double getAverage(int *arr, int size);

int main () {
    int a[5] = {8, 2, 3, 17, 50};
    double avg;
    avg = getAverage(a, 5);
    printf("Average value is: %f\n", avg);
    return 0;
}
```

Passing an one-dimensional array to the function as a pointer (2/2)

```
double getAverage(int *arr, int size) {  
    int i, sum = 0;  
    double avg;  
    for (i=0; i<size; ++i) {  
        sum += *(arr+i);  
    }  
    avg = (double)sum / size;  
    return avg;  
}
```

The operator *return* (1/2)

The return value of the function is specified in the operator **return**

An operand of the operator return can be any expression, whose result matches the type of the return value

For example:

- `abscmp(double a, double b) {
 return fabs(a) < fabs(b);
}`
- two real numbers are compared, so the result of comparison will be 0 or 1
- that's why by default the type of returned value will be *int*, even if it's not specified in declaration of function

The operator *return* (2/2)

```
#include <stdio.h>
#include <math.h>
int main() {
    double a = 3, b = -5;
    printf("Result = %d", abscmp(a, b));
    return 0;
}
abscmp(double a, double b) {
    return fabs(a) < fabs(b);
}
```

Function *main* () (1/2)

The **main()** function is called at program startup after initialization of the non-local objects with static storage duration

It is the designated entry point to a program that is executed in hosted environment (that is, with an operating system)

Declarations:

- `int main () { body }`
- `int main (int argc, char *argv[]) { body }`
- `int main (int argc, char *argv[] , other_parameters ) { body }`

Function *main* () (2/2)

The main function has several **special properties**:

- It cannot be used anywhere in the program
- in particular, it cannot be called recursively
- its address cannot be taken

The body of the main function **does not need to contain the return statement**:

- if control reaches the end of main without encountering a return statement, the effect is the same as execution of *return 0*;

Global and local variables (1/2)

A **local variable** is a variable that is declared inside a function, so it can only be used in the function where it is declared

A **global variable** is a variable that is declared outside all functions, so it can be used in all functions

```
#include<stdio.h>
// Global variables
int a; int b;
int add() {
    return a + b;
}
```

Global and local variables (2/2)

```
int main() {  
    // Local variable  
    int answer;  
    a = 5;  
    b = 7;  
    answer = add();  
    printf("%d\n", answer);  
    return 0;  
}
```